RabbitMQ Interview Notes (Official Tutorials – Hinglish)

> **Scope:** Ye document RabbitMQ ke official Python (Pika) tutorials se banaya gaya hai – *Hello World → Work Queues → Routing → Topics → RPC*.
> **Use:** Interview prep + quick revision + real-world mapping.

---

1. RabbitMQ Basics (Hello World)

RabbitMQ kya hai?

- **Message Broker**: Producer se message leta hai, queue me store karta hai, consumer ko deliver karta hai.
- Post office analogy: Producer = letter sender, Queue = post box, Consumer = receiver.

Core Terms

- **Producer**: Message bhejne wala app
- **Queue**: Message buffer (memory/disk based)
- **Consumer**: Message receive/process karne wala app
- **Exchange**: Message routing logic (producer direct queue me nahi bhejta)

Default Exchange

- Name: `""` (empty string)
- Rule: `routing_key == queue_name`

Hello World Flow

```
Producer → default exchange → queue(hello) → Consumer
```

Interview Points

- Message direct queue me nahi jaata, exchange ke through jaata hai
- `queue_declare` idempotent hota hai

---

2. Work Queues (Task Queues)

Problem Statement

- Heavy kaam (PDF, email, invoice) HTTP request ke time pe nahi karna

- Background me async process karna

Solution

- Task ko message bana ke **work queue** me daal do
- Multiple **workers** parallel me kaam karte hain

```
Producer → task_queue → Worker1 / Worker2 / Worker3
```

Key Concepts

(a) Round-robin Dispatch (Default)

- RabbitMQ messages consumers ko sequence me deta hai
- Load ka idea nahi hota

(b) Message Acknowledgement (ACK)

- Consumer ka signal: "Message process ho gaya"
- ACK nahi aaya → message re-queue

```
ch.basic_ack(delivery_tag=method.delivery_tag)
```

⚠️ACK bhool gaye → memory leak + duplicate delivery

(c) Durability

- **Queue durable**: server restart me queue rahe
- **Message persistent**: disk pe likha jaaye

```
queue_declare(queue='task_queue', durable=True)
properties=pika.BasicProperties(delivery_mode=2)
```

(d) Fair Dispatch

- Problem: ek worker heavy tasks me busy, dusra idle
- Solution: `prefetch_count=1`

```
channel.basic_qos(prefetch_count=1)
```

Interview Points

- Work queue = async background processing
- ACK + durable queue + persistent message = reliability
- `prefetch_count=1` ensures fair load

3. Routing (Direct Exchange)

Problem

- Fanout exchange sabko message bhejta hai
- Chahiye selective delivery (e.g., sirf error logs)

Direct Exchange

- Rule: `routing_key == binding_key`

Binding

- Queue bolti hai: mujhe kaunse routing keys chahiye

```
queue_bind(exchange='direct_logs', queue=q, routing_key='error')
```

Multiple Bindings

- Ek queue multiple routing keys se bind ho sakti hai
- Same routing key multiple queues me ho sakti hai

Use Case

- Logging system: `info`, `warning`, `error`

Interview Points

- Direct exchange exact match pe kaam karta hai
- Routing producer decide karta hai

---

4. Topics (Topic Exchange)

Problem

- Direct exchange sirf single criteria handle karta hai
- Real systems me multi-criteria routing chahiye

Topic Exchange

- Routing key format: `word.word.word`
- Example: `kern.critical`, `order.created.india`

Wildcards

- `*` → exactly one word
- `#` → zero or more words

Example Bindings

- `kern.*` → kern ke saare logs
- `*.critical` → sab critical logs
- `#` → sab messages (fanout jaisa)

Behaviour

- No wildcard → direct jaisa
- `#` only → fanout jaisa

Interview Points

- Topic exchange most powerful exchange
- Pattern-based routing karta hai

---

5. RPC (Remote Procedure Call)

RPC kya hai?

- Remote machine pe function call karna aur result wait karna

```
result = rpc_client.call(30)
```

⚠️Blocking nature – misuse dangerous

RPC Flow

```
Client → rpc_queue → Server
Client ← callback_queue ← Server
```

Important Concepts

(a) Callback Queue

- Client apni temporary exclusive queue banata hai

(b) reply_to

- Server ko batata hai response kahan bhejna hai

(c) correlation_id

- Request–response match karne ke liye unique ID

Why correlation_id?

- Ek client multiple requests bhej sakta hai
- Responses mix ho sakte hain

Scaling

- Multiple RPC servers chala ke scale kar sakte ho

Interview Warnings

- RPC ko async pipelines ke badle prefer mat karo
- Timeout, retry, idempotency handle karni chahiye

---

6. Comparison Cheat Sheet

Pattern / Exchange	Use Case
Fanout	Broadcast
Direct	Simple routing
Topic	Complex routing
Work Queue	Background jobs
RPC	Request/Response (careful!)

---

7. Final Interview One-Liners

- RabbitMQ producer se message leta hai, exchange route karta hai
- ACK ensures no message loss
- Durable queue + persistent message = crash safety
- `prefetch_count=1` = fair dispatch
- Topic exchange supports wildcard routing
- RPC uses `reply_to` + `correlation_id` but is blocking

Tip: Interview me hamesha ye bolo ki *"RPC ke badle async event-driven approach zyada scalable hota hai"* – ye senior-level signal hai.